04 HAFTA

C# ile Nesneye Yönelik Programlama · Ders Notu

İleri C# ve Bellek Yönetimi

(Dilin Derinlikleri)

Bu Haftanın Kazanımları

Bu dersin sonunda öğrenciler:

- ✓ **value type (değer tipi)** ile **reference type (referans tipi)** arasındaki farkı açıklar; `struct` ile `class` arasında bilinçli seçim yapar.
- ✓ `enum` ve `[Flags]` ile anlamlı sabit kümeleri tanımlar; **tuple (demet)** ile metottan birden fazla değer döndürür.
- ✓ **nullable value type (boş olabilen değer tipi)** ve **nullable reference type (boş olabilen referans tipi)** kavramlarını uygular, null operatörlerini doğru kullanır.
- ✓ **stack (yığın)** ve **heap (öbek)** bölgelerini ve **garbage collector (çöp toplayıcı)** mantığını açıklar.
- ✓ `IDisposable` arabirimi ve **Dispose pattern (Dispose kalıbı)** ile yönetilmeyen kaynakları güvenle serbest bırakır.

İçindekiler

1. Struct ve Class Farkları
2. Enum ve Tuple Kullanımı
3. Nullable Types ve Null Kontrolleri
4. Stack vs Heap Hafıza Yönetimi
5. Dispose Pattern, IDisposable Arabirimi

— Haftanın Özeti

— Anahtar Kavramlar

- Alıştırmalar
- Ek Okuma Kaynakları

1. Struct ve Class Farkları

C#'ta her tip iki büyük gruptan birine aittir: **value type (değer tipi)** veya **reference type (referans tipi)**. `int`, `double`, `bool`, `DateTime`, tüm `enum`'lar ve kendi yazdığımız her `struct` birer değer tipidir. `class`, `string`, diziler, `delegate`'ler ve `record` (class) ise referans tipidir. Bu ayrım; atama yaptığınızda, metoda parametre geçirdiğinizde ve iki nesneyi karşılaştırdığınızda davranışı tamamen değiştirir.

1.1 Kopyalama davranışı: değer mi, referans mı?

Değer tipindeki bir değişken verinin kendisini taşır; atama yapıldığında verinin tamamı kopyalanır. Referans tipindeki bir değişken ise nesnenin bellekteki adresini, yani **reference (referans)** bilgisini taşır; atamada yalnızca bu adres kopyalanır ve iki değişken aynı nesneyi gösterir.

C# · Program.cs

```
var s1 = new NoktaS { X = 1, Y = 1 };
var s2 = s1;               // struct: verinin TAMAMI kopyalanır
s2.X = 99;
Console.WriteLine(s1.X);   // 1  -> s1 etkilenmedi

var c1 = new NoktaC { X = 1, Y = 1 };
var c2 = c1;               // class: yalnızca referans (adres) kopyalanır
c2.X = 99;
Console.WriteLine(c1.X);   // 99 -> ikisi de aynı nesneyi gösteriyor

// Tip bildirimleri, top-level statement'lardan SONRA yazılmalıdır
public struct NoktaS { public int X; public int Y; }   // değer tipi
public class NoktaC { public int X; public int Y; }    // referans tipi
```

Aynı kural metot parametrelerinde de geçerlidir. Bir `struct` metoda geçirildiğinde metot bir kopya üzerinde çalışır; metodun yaptığı değişiklik çağıran tarafı etkilemez. Bir `class` nesnesi geçirildiğinde ise referansın kopyası gider; metot nesnenin alanlarını değiştirirse çağıran taraf da bu değişikliği görür.

1.2 struct'ın özellikleri ve kısıtları

- Tüm `struct`'lar örtük olarak `System.ValueType`'tan türer. Başka bir tipten **inheritance (kalıtım)** alamaz, kendisinden de tip türetilemez (örtük olarak `sealed`'dır).
- **interface (arabirim)** uygulayabilir: `public struct Para : IComparable<Para>`.
- C# 10'dan itibaren parametresiz **constructor (yapıcı metot)** ve alan başlatıcıları yazılabilir. Ancak `default(T)` her zaman tüm alanları sıfırlanmış bir örnek üretir; constructor çalışmaz.
- Doğrudan `null` olamaz; boş olabilmesi için nullable hâli (`Para?`) kullanılır.
- Varsayılan `Equals` metodu alanları tek tek karşılaştırır, fakat bunu çoğu durumda **reflection (yansıma)** ile yaptığı için yavaştır. `IEquatable<T>` uygulamak veya `record struct` kullanmak önerilir.

1.3 readonly struct ve record struct

Değer tiplerinde en güvenli yaklaşım **immutable (değişmez)** tasarımdır. `readonly struct` tüm alanların salt okunur olmasını derleyiciye garanti ettirir; `record struct` ise değer eşitliği, `ToString()` ve `with` ifadesini otomatik üretir.

C# · Para.cs

```csharp
// readonly struct: oluşturulduktan sonra hiçbir alanı değişmez
public readonly struct Para
{
    public decimal Tutar { get; }
    public string ParaBirimi { get; }

    public Para(decimal tutar, string paraBirimi)
    {
        Tutar = tutar;
        ParaBirimi = paraBirimi;
    }

    public Para Topla(Para diger) =>
        diger.ParaBirimi == ParaBirimi
            ? new Para(Tutar + diger.Tutar, ParaBirimi)
            : throw new InvalidOperationException("Para birimleri farklı.");

    public override string ToString() => $"{Tutar:N2} {ParaBirimi}";
}

// record struct: eşitlik, ToString ve 'with' derleyiciden hazır gelir
public readonly record struct Koordinat(double Enlem, double Boylam);
```

C# · Program.cs

```csharp
var fiyat = new Para(150m, "TRY").Topla(new Para(49.90m, "TRY"));
Console.WriteLine(fiyat);                    // 199,90 TRY

var a = new Koordinat(41.01, 28.97);
var b = a with { Boylam = 29.03 };           // a değişmez, yeni bir kopya oluşur
Console.WriteLine(a == new Koordinat(41.01, 28.97));   // True (değer eşitliği)
```

1.4 Ne zaman struct, ne zaman class?

Özellik	`struct`	`class`
Tip kategorisi	value type (değer tipi)	reference type (referans tipi)
Atama / parametre geçişi	Verinin tamamı kopyalanır	Yalnızca referans kopyalanır
`null` olabilir mi?	Hayır (`T?` hariç)	Evet
Kalıtım	Yok (interface uygulayabilir)	Var (tek bir sınıftan)
Varsayılan eşitlik	Alan değerleri (yavaş olabilir)	Referans eşitliği (aynı nesne mi?)
Bellekteki tipik yeri	Tanımlandığı yer (stack ya da içinde bulunduğu nesne)	Heap
GC maliyeti	Tek başına yok	Var
Uygun kullanım	Küçük, kısa ömürlü, değişmez değerler	Kimliği ve yaşam döngüsü olan varlıklar

İPUCU **Microsoft'un struct yönergesi**

Tip mantıksal olarak tek bir değeri temsil ediyorsa (`Para`, `Koordinat`, `RenkRgb` gibi), örnekleri küçükse (yaklaşık 16 bayt altı), değişmezse ve sık sık **boxing (kutulama)** gerektirmeyecekse `struct` düşünün. Diğer tüm durumlarda varsayılan tercih `class`'tır.

2. Enum ve Tuple Kullanımı

2.1 enum: anlamlı sabit kümeleri

enum (numaralandırma), isimlendirilmiş sabitlerden oluşan bir değer tipidir. Varsayılan altta yatan tip `int`'tir ve değerler 0'dan başlar. Kodda `if (durum == 3)` gibi anlamı belirsiz **magic number (sihirli sayı)** kullanmak yerine `if (durum == SiparisDurumu.Kargoda)` yazmak hem okunabilirliği hem de derleyici desteğini artırır.

C# · SiparisDurumu.cs

```
public enum SiparisDurumu
{
    Beklemede,          // 0
    Onaylandi,          // 1
    Kargoda = 10,       // değer açıkça verilebilir
    TeslimEdildi,       // 11 (bir öncekinden devam eder)
    IptalEdildi = -1
}

// Altta yatan tip değiştirilebilir (bellekten tasarruf için)
public enum Oncelik : byte { Dusuk = 1, Orta = 2, Yuksek = 3 }
```

C# · Program.cs

```
var durum = SiparisDurumu.Kargoda;
int kod = (int)durum;                  // 10
string ad = durum.ToString();          // "Kargoda"

// string -> enum: Parse hata fırlatabilir, TryParse güvenlidir
if (Enum.TryParse<SiparisDurumu>("Onaylandi", out var sonuc))
    Console.WriteLine(sonuc);          // Onaylandi

foreach (var d in Enum.GetValues<SiparisDurumu>())
    Console.WriteLine($"{(int)d,3} -> {d}");

// switch expression ile enum'a göre karar
string Aciklama(SiparisDurumu d) => d switch
{
    SiparisDurumu.Beklemede   => "Ödeme bekleniyor",
    SiparisDurumu.Kargoda     => "Yolda",
    SiparisDurumu.IptalEdildi => "İptal edildi",
    _                         => "Diğer"
};
```

> **DİKKAT** **enum değerleri doğrulanmaz**
>
> `(SiparisDurumu)999` ifadesi derlenir ve çalışır; enum, tanımlı olmayan bir sayıyı tutmanızı engellemez. Dışarıdan (kullanıcıdan, veritabanından, API'den) gelen değerleri `Enum.IsDefined(deger)` ile kontrol edin.

2.2 [Flags] ile bit bayrakları

Bir değişkenin aynı anda birden fazla seçeneği taşıması gerekiyorsa (ör. kullanıcı izinleri) `[Flags]` **attribute (öznitelik)** ile işaretlenmiş, değerleri 2'nin kuvvetleri olan bir enum kullanılır. Her değer tek bir biti temsil eder ve **bitwise (bit düzeyinde)** operatörlerle birleştirilir.

C# · Izin.cs

```
[Flags]
public enum Izin
{
    Yok     = 0,
    Okuma   = 1 << 0,                  // 1
    Yazma   = 1 << 1,                  // 2
    Silme   = 1 << 2,                  // 4
    Yonetim = Okuma | Yazma | Silme    // 7
}
```

C# · Program.cs

```
var izin = Izin.Okuma | Izin.Yazma;          // 3
Console.WriteLine(izin);                      // "Okuma, Yazma"
bool yazabilir = izin.HasFlag(Izin.Yazma);    // true
izin |= Izin.Silme;                           // bayrak ekle
izin &= ~Izin.Yazma;                          // bayrak kaldır
bool silebilir = (izin & Izin.Silme) != 0;    // HasFlag'in klasik karşılığı
```

2.3 Tuple: birden fazla değeri bir arada taşımak

tuple (demet), birkaç değeri ayrı bir tip tanımlamadan tek bir yapıda taşımanızı sağlar. C#'ta iki tuple türü vardır: eski `System.Tuple<T1, T2>` (referans tipi, elemanları `Item1`, `Item2`) ve C# 7 ile gelen `System.ValueTuple` (değer tipi, elemanlarına isim verilebilir, dilin kendi sözdizimiyle yazılır). Modern kodda neredeyse her zaman `ValueTuple` kullanılır.

C# · TupleOrnekleri.cs

```
// 1) Metottan birden fazla değer döndürmek
(double Ortalama, int Min, int Max) Istatistik(int[] notlar)
    => (notlar.Average(), notlar.Min(), notlar.Max());

var s = Istatistik([70, 85, 92, 64]);        // C# 12 collection expression
Console.WriteLine($"Ort: {s.Ortalama:F1}, Min: {s.Min}, Max: {s.Max}");

// 2) Deconstruction (ayrıştırma): tuple'ı doğrudan değişkenlere açmak
var (ort, enDusuk, _) = Istatistik([50, 60, 70]);   // _ : discard (yok say)

// 3) İki değişkenin yerini tek satırda değiştirmek (swap)
int a = 1, b = 2;
(a, b) = (b, a);

// 4) Tuple eşitliği: elemanlar sırayla karşılaştırılır
var kisi = (Ad: "Ali", Yas: 20);
Console.WriteLine(kisi == ("Ali", 20));      // True
```

	`Tuple<T1, T2>`	`(T1, T2)` ValueTuple	`record`
Tip kategorisi	Referans (class)	Değer (struct)	Referans (varsayılan)
Eleman adları	`Item1`, `Item2`	İsim verilebilir	Property adları
Değiştirilebilir mi?	Hayır	Evet (alanlar)	Hayır (`init`)
Tipik kullanım	Eski kod tabanları	Kısa ömürlü, yerel gruplama	Public API, domain modeli

> **İPUCU** **Tuple mı, record mı?**
>
> Tuple eleman adları yalnızca derleme zamanında vardır; reflection veya JSON serileştirmede kaybolur. Bir metodun içinde ya da `private` yardımcılarda tuple idealdir. Public bir API'de veya uzun yaşayacak verilerde anlamlı isimli bir `record` tanımlayın.

3. Nullable Types ve Null Kontrolleri

3.1 Nullable value types

Değer tipleri normalde `null` olamaz; ancak veritabanında NULL olabilen bir sütun veya doldurulması zorunlu olmayan bir form alanı gibi durumlarda "değer yok" bilgisini ifade etmemiz gerekir. `int?` yazımı, `Nullable<int>` yapısının kısaltmasıdır ve **syntactic sugar (sözdizimsel kolaylık)** olarak derlenir.

C# · Program.cs

```
int? stok = null;
Console.WriteLine(stok.HasValue);          // False
int guvenli = stok.GetValueOrDefault();    // 0
int varsayilan = stok ?? -1;               // -1
// int hata = stok.Value;                  // InvalidOperationException!

// Pattern matching: null değilse değeri yeni bir değişkene açar
if (stok is int adet)
    Console.WriteLine($"Stokta {adet} ürün var");
```

3.2 Null operatörleri

Operatör	Adı	Örnek	Anlamı
`?.`	null-conditional (null koşullu)	`musteri?.Adres?.Sehir`	Sol taraf `null` ise zincir durur, sonuç `null` olur
`?[]`	null-conditional index	`liste?[0]`	Koleksiyon `null` ise `null` döner
`??`	null-coalescing (null birleştirme)	`ad ?? "Misafir"`	Sol taraf `null` ise sağdaki değeri kullanır
`??=`	null-coalescing assignment	`onbellek ??= Yukle()`	Yalnızca `null` ise atama yapar
`!`	null-forgiving (null affedici)	`ad!.Length`	Derleyici uyarısını susturur; çalışma zamanında koruma sağlamaz
`is null`	null pattern	`if (x is null)`	Aşırı yüklenmiş `==` operatöründen etkilenmez

C#

```
string sehir = siparis?.Musteri?.Adres?.Sehir ?? "Bilinmiyor";
List<string> etiketler = urun.Etiketler ??= [];   // null ise boş liste ata

if (siparis is not null && siparis.Tutar > 1000m)
    Console.WriteLine("Kargo ücretsiz");
```

3.3 Nullable reference types (NRT)

C# 8 ile gelen **nullable reference types (boş olabilen referans tipleri)** özelliği, yeni oluşturulan tüm .NET projelerinde `.csproj` dosyasındaki `<Nullable>enable</Nullable>` ayarıyla varsayılan olarak açıktır. Bu ayar açıkken `string` "asla null olmayan", `string?` ise "null olabilen" bir referans anlamına gelir. Derleyici **static flow analysis (statik akış analizi)** yapar ve olası null hatalarını daha kod çalışmadan uyarı olarak bildirir (CS8600, CS8602, CS8618 gibi).

XML · OkulUygulamasi.csproj

```xml
<PropertyGroup>
  <OutputType>Exe</OutputType>
  <TargetFramework>net10.0</TargetFramework>
  <Nullable>enable</Nullable>
  <ImplicitUsings>enable</ImplicitUsings>
</PropertyGroup>
```

C# · Ogrenci.cs

```csharp
public class Ogrenci
{
    public required string Ad { get; init; }     // C# 11: oluştururken zorunlu
    public string? IkinciAd { get; set; }        // null olabilir
    public List<string> Dersler { get; } = [];   // asla null değil

    public int IkinciAdUzunlugu()
    {
        // return IkinciAd.Length;               // CS8602: olası null başvurusu
        return IkinciAd?.Length ?? 0;            // güvenli
    }
}

public static class OgrenciServisi
{
    public static void Kaydet(Ogrenci ogrenci)
    {
        // Guard clause: hatalı girdiyi metodun en başında reddet
        ArgumentNullException.ThrowIfNull(ogrenci);
        ArgumentException.ThrowIfNullOrWhiteSpace(ogrenci.Ad);
        // ... kayıt işlemleri
    }
}
```

YENİ **C# 14: null-conditional assignment**

C# 14 ile `?.` operatörü atamanın sol tarafında da kullanılabilir. Alıcı nesne `null` ise atama yapılmaz ve sağ taraftaki ifade hiç çalıştırılmaz. `+=` gibi bileşik atamalar da desteklenir; `++` ve `--` desteklenmez.

C#

```csharp
// C# 14 öncesi
if (musteri is not null)
    musteri.SonGiris = DateTime.Now;

// C# 14 ile
musteri?.SonGiris = DateTime.Now;
sepet?.ToplamTutar += urun.Fiyat;
```

> **DİKKAT** `!` **operatörü bir çözüm değildir**
>
> `ad!.Length` yazmak yalnızca derleyiciye "bana güven, bu null değil" demektir. Değer gerçekten `null` ise çalışma zamanında yine `NullReferenceException` alırsınız. `!` operatörünü, değerin null olmadığını sizin kanıtlayabildiğiniz ama derleyicinin anlayamadığı nadir durumlara saklayın.

4. Stack vs Heap Hafıza Yönetimi

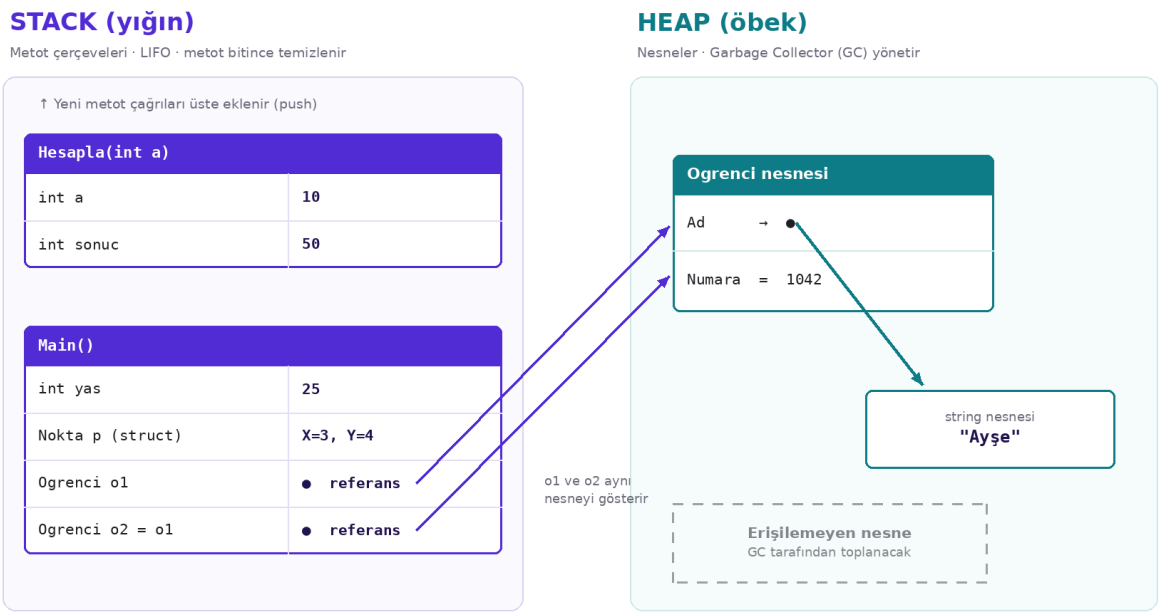
.NET uygulamaları belleği iki farklı bölgede yönetir: **stack (yığın)** ve **managed heap (yönetilen öbek)**. Bu terimler Türkçe kaynaklarda farklı karşılıklarla geçebildiği için sektörde genellikle İngilizce kullanılır.

4.1 Stack nedir?

- Her **thread (iş parçacığı)** kendi stack'ine sahiptir ve boyutu sınırlıdır (Windows'ta varsayılan yaklaşık 1 MB).
- **LIFO (Last In, First Out — son giren ilk çıkar)** mantığıyla çalışır. Her metot çağrısında parametreleri, yerel değişkenleri ve dönüş adresini içeren bir **stack frame (yığın çerçevesi)** eklenir.
- Bellek ayırma ve serbest bırakma yalnızca bir işaretçinin kaydırılmasıdır; bu yüzden son derece hızlıdır. Metot bittiğinde çerçeve kendiliğinden kaldırılır.
- Sonsuz ya da çok derin **recursion (özyineleme)** stack'i doldurur ve `StackOverflowException` oluşur. Bu hata yakalanamaz; süreç sonlanır.

4.2 Heap nedir?

- Tüm thread'lerin paylaştığı büyük bir bellek alanıdır. Referans tipi nesneler `new` ile burada oluşturulur.
- Nesnenin ömrü metoda bağlı değildir. Bir **root (kök)** üzerinden (yerel değişken, statik alan, CPU yazmacı vb.) erişilebildiği sürece yaşar.
- Erişilemez hâle gelen nesnelerin belleği, **garbage collector (çöp toplayıcı)** tarafından geri kazanılır.



Şekil 1. *Bir metot çağrısı sırasında stack ve heap'in durumu; referans değişkenler heap'teki nesneleri gösterir*

Şekildeki durumu oluşturan kod aşağıdadır. Dikkat edin: `Nokta` bir `struct` olduğu için `new` ile oluşturulsa bile değeri doğrudan stack çerçevesinde durur; `new` anahtar kelimesi tek başına "heap" anlamına gelmez.

C# · BellekOrnegi.cs

```
int yas = 25;                          // değer doğrudan Main çerçevesinde
var p = new Nokta(3, 4);               // struct: değer yine çerçevede
var o1 = new Ogrenci("Ayşe", 1042);    // nesne heap'te, o1 yalnızca bir referans
var o2 = o1;                           // aynı nesneyi gösteren ikinci referans
Console.WriteLine(Hesapla(10));

static int Hesapla(int a)
{
    int sonuc = a * 5;                 // yeni çerçeve: a ve sonuc
    return sonuc;                      // metot bitince çerçeve kaldırılır
}

record struct Nokta(int X, int Y);
record Ogrenci(string Ad, int Numara);
```

BİLGİ "Değer tipleri stack'te yaşar" tam olarak doğru değildir

Bir değerin nerede duracağını tipi değil, **nerede tanımlandığı** belirler:

- Yerel değer tipi değişkenler genellikle stack'tedir.
- Bir `class`'ın içindeki `int` alan, o nesneyle birlikte heap'tedir.
- `Nokta[]` dizisi heap'te bir nesnedir; `struct` elemanlar dizinin içinde yan yana durur.
- Lambda ifadelerinin yakaladığı ya da `await` boyunca yaşaması gereken yerel değişkenler, derleyicinin ürettiği bir sınıfa taşınarak heap'e çıkar.

4.3 Garbage Collector nasıl çalışır?

.NET GC'si **generational (nesil tabanlı)** çalışır. Temel gözlem şudur: nesnelerin büyük çoğunluğu kısa ömürlüdür. GC önce kök referanslardan başlayarak erişilebilen tüm nesneleri işaretler (**mark**), erişilemeyenlerin belleğini geri alır ve yaşayan nesneleri bir araya kaydırarak parçalanmayı giderir (**compact — sıkıştırma**). Bir toplamadan sağ çıkan nesne bir üst nesle terfi eder.

Nesil	İçerik	Toplanma sıklığı
Gen 0	Yeni oluşturulan, kısa ömürlü nesneler	Çok sık, çok hızlı
Gen 1	Gen 0 toplamasından sağ çıkanlar (tampon bölge)	Orta
Gen 2	Uzun ömürlü nesneler (önbellek, singleton, statik veriler)	Seyrek, maliyetli
LOH	85.000 bayt ve üzeri nesneler (**Large Object Heap**)	Gen 2 ile birlikte; varsayılan olarak sıkıştırılmaz

C# · GcGozlem.cs

```
var nesne = new byte[1_000];
Console.WriteLine(GC.GetGeneration(nesne));   // 0

GC.Collect();                                  // SADECE deney amaçlı!
Console.WriteLine(GC.GetGeneration(nesne));   // 1 -> sağ çıktı, terfi etti

var buyuk = new byte[100_000];                 // 85.000 bayt üzeri -> LOH
Console.WriteLine(GC.GetGeneration(buyuk));   // 2 -> LOH mantıksal olarak Gen 2

Console.WriteLine($"Toplam: {GC.GetTotalMemory(false) / 1024} KB");
Console.WriteLine($"Gen0 sayısı: {GC.CollectionCount(0)}");
```

DİKKAT **GC.Collect() çağırmayın**

GC ne zaman çalışacağına kendi istatistiklerine göre karar verir ve bu kararlarda genellikle sizden daha isabetlidir. Üretim kodunda `GC.Collect()` çağırmak, genç nesneleri gereksiz yere yaşlı nesillere terfi ettirerek performansı çoğu zaman düşürür.

4.4 Boxing ve unboxing

boxing (kutulama), bir değer tipinin `object` veya bir interface tipine dönüştürülmesidir. Bu sırada heap'te yeni bir nesne ayrılır ve değer içine kopyalanır. **unboxing (kutudan çıkarma)** ise açık bir **cast (tip dönüşümü)** ile değeri geri almaktır; tip kontrolü ve yeniden kopyalama içerir. Döngü içinde fark edilmeden yapılan kutulama, binlerce gereksiz heap nesnesi ve GC yükü demektir.

C# · Program.cs

```
int sayi = 42;
object kutu = sayi;          // boxing: heap'te yeni nesne, değer kopyalandı
int geri = (int)kutu;        // unboxing: tip kontrolü + kopyalama
// long yanlis = (long)kutu; // InvalidCastException: kutudaki tip int!

var eski = new System.Collections.ArrayList();
for (int i = 0; i < 1000; i++) eski.Add(i);   // 1000 boxing = 1000 heap nesnesi

var yeni = new List<int>();
for (int i = 0; i < 1000; i++) yeni.Add(i);   // generic: boxing yok
```

4.5 ref, out ve in parametreleri

Değer tiplerini metotlara kopyalamadan, doğrudan değişkenin kendisini geçirerek aktarmak için üç anahtar kelime vardır:

Anahtar kelime	Ne yapar?	Çağıran tarafta
`ref`	Değişkenin kendisini geçirir; metot okuyabilir ve değiştirebilir	Değişkene önceden değer atanmış olmalı
`out`	Metot değişkene değer atamak zorundadır; çoklu dönüşte kullanılır	Atanmamış olabilir (`out var x`)
`in`	Salt okunur referans; büyük struct'ları kopyasız geçirmek için	Metot değeri değiştiremez

C# · Program.cs

```
int sayac = 5;
Arttir(ref sayac);                                    // sayac artık 6
bool basarili = int.TryParse("123", out int deger);   // out: ikinci çıktı

static void Arttir(ref int x) => x++;
```

5. Dispose Pattern, IDisposable Arabirimi

5.1 Yönetilen ve yönetilmeyen kaynaklar

GC yalnızca yönetilen belleği tanır. Dosya tanıtıcıları, veritabanı bağlantıları, ağ soketleri ve işletim sistemi nesneleri gibi **unmanaged resources (yönetilmeyen kaynaklar)** hakkında bilgisi yoktur. Bu kaynakların GC'nin ne zaman çalışacağını beklemeden, **deterministic (belirlenimci)** biçimde, yani tam istediğimiz anda serbest bırakılması gerekir. .NET'te bunun standart sözleşmesi tek metotlu `IDisposable` arabirimidir: `void Dispose()`.

5.2 using ifadesi ve using bildirimi

`using` bloğu, blok sona erdiğinde (hata oluşsa bile) nesnenin `Dispose()` metodunu otomatik olarak çağırır. C# 8 ile gelen **using declaration (using bildirimi)** ise süslü parantez gerektirmez; nesne, içinde bulunduğu kapsamın sonunda Dispose edilir.

C# · Program.cs

```
// 1) Klasik using bloğu
using (var okuyucu = new StreamReader("notlar.txt"))
{
    Console.WriteLine(okuyucu.ReadLine());
}   // <- okuyucu.Dispose() burada, hata olsa bile çağrılır

// 2) using bildirimi (C# 8): kapsamın sonunda Dispose edilir
using var yazici = new StreamWriter("rapor.txt");
yazici.WriteLine("Merhaba");
```

Derleyici `using` bloğunu arka planda bir `try/finally` yapısına dönüştürür:

C# · Derleyicinin ürettiği karşılık

```
var okuyucu = new StreamReader("notlar.txt");
try
{
    Console.WriteLine(okuyucu.ReadLine());
}
finally
{
    if (okuyucu != null) ((IDisposable)okuyucu).Dispose();
}
```

5.3 Kendi IDisposable sınıfımızı yazmak

En sık karşılaşılan durum, sınıfınızın içinde başka `IDisposable` nesneler tutmasıdır. Sınıf `sealed` ise ve doğrudan yönetilmeyen bir kaynağa sahip değilse basit bir `Dispose()` yeterlidir:

C# · RaporYazici.cs

```
public sealed class RaporYazici : IDisposable
{
    private readonly StreamWriter _yazici;
    private bool _disposed;

    public RaporYazici(string dosyaYolu) => _yazici = new StreamWriter(dosyaYolu);

    public void SatirEkle(string metin)
    {
        ObjectDisposedException.ThrowIf(_disposed, this);   // .NET 7+
        _yazici.WriteLine(metin);
    }

    public void Dispose()
    {
        if (_disposed) return;     // Dispose birden fazla kez çağrılabilmeli
        _yazici.Dispose();
        _disposed = true;
    }
}
```

5.4 Tam Dispose kalıbı: Dispose(bool) ve finalizer

Sınıf doğrudan yönetilmeyen bir kaynağa (ör. `IntPtr` ile tutulan bir işletim sistemi tanıtıcısına) sahipse ve kalıtıma açıksa, Microsoft'un önerdiği tam kalıp kullanılır. Kalıbın merkezinde `protected virtual void Dispose(bool disposing)` metodu ve son güvenlik ağı olarak bir **finalizer (sonlandırıcı)** bulunur.

C# · KaynakYoneticisi.cs

```
public class KaynakYoneticisi : IDisposable
{
    private IntPtr _tutamac;                  // yönetilmeyen kaynak (OS handle)
    private MemoryStream? _tampon = new();    // yönetilen ama IDisposable kaynak
    private bool _disposed;

    public void Dispose()
    {
        Dispose(disposing: true);
        GC.SuppressFinalize(this);            // finalizer'a artık gerek yok
    }

    protected virtual void Dispose(bool disposing)
    {
        if (_disposed) return;

        if (disposing)
        {
            // Yönetilen kaynaklar: YALNIZCA Dispose() ile çağrıldığında
            _tampon?.Dispose();
            _tampon = null;
        }
```

C# · KaynakYoneticisi.cs

```
        // Yönetilmeyen kaynaklar: her iki durumda da serbest bırakılır
        if (_tutamac != IntPtr.Zero)
        {
            NativeKapat(_tutamac);
            _tutamac = IntPtr.Zero;
        }
        _disposed = true;
    }

    ~KaynakYoneticisi() => Dispose(disposing: false);   // son güvenlik ağı

    private static void NativeKapat(IntPtr tutamac) { /* P/Invoke çağrısı */ }
}
```

`disposing` parametresi metodun kimin tarafından çağrıldığını belirtir. `true` ise çağrı kullanıcı kodundan (`Dispose()`) gelmiştir ve yönetilen nesnelere dokunmak güvenlidir. `false` ise çağrı finalizer thread'inden gelmiştir; bu noktada diğer yönetilen nesneler zaten sonlandırılmış olabileceği için yalnızca yönetilmeyen kaynaklar temizlenir.

> **İPUCU** **Kendi finalizer'ınızı yazmak yerine SafeHandle**
>
> Finalizer'ı olan nesneler en az bir GC turu daha yaşar ve performansı düşürür. Yönetilmeyen tanıtıcıları ham `IntPtr` yerine `SafeHandle` türevleriyle (`SafeFileHandle` gibi) sarmalamak, finalizer mantığını güvenilir biçimde .NET'e bırakmanızı sağlar.

5.5 IAsyncDisposable ve await using

Kapatılması sırasında G/Ç işlemi yapan kaynaklar (dosya akışları, veritabanı bağlantıları, EF Core `DbContext`) `IAsyncDisposable` arabirimini de uygular. Bu nesneler `await using` ile kullanılarak kapanış işlemi thread bloklanmadan yapılır. Asenkron programlama 10. haftada ayrıntılı işlenecektir.

C# · Program.cs

```
await using var dosya = File.Create("yedek.bin");
await dosya.WriteAsync(new byte[] { 1, 2, 3 });
// kapsam sonunda: await dosya.DisposeAsync();
```

5.6 Sık yapılan hatalar

Hata	Sonuç	Doğrusu
`Dispose()` çağırmayı unutmak	Dosya kilitli kalır, bağlantı havuzu tükenir	`using` / `await using` kullanın
Dependency Injection ile gelen nesneyi Dispose etmek	Aynı nesneyi kullanan diğer sınıflar hata alır	Nesneyi oluşturan (DI container) Dispose eder
Her istekte `new HttpClient()` + `using`	**Socket exhaustion (soket tükenmesi)**	`IHttpClientFactory` veya paylaşılan tek örnek
Finalizer içinde yönetilen nesnelere dokunmak	Zaten sonlandırılmış nesne, öngörülemeyen hata	`disposing == false` iken yalnızca yönetilmeyen kaynak
Gereksiz yere finalizer yazmak	Nesne fazladan GC turu yaşar	Yalnızca doğrudan yönetilmeyen kaynak varsa

Haftanın Özeti

- `struct` bir değer tipidir ve atamada kopyalanır; `class` bir referans tipidir ve atamada yalnızca adres kopyalanır. Küçük, değişmez, tek bir değeri temsil eden tipler için `readonly struct` veya `record struct` uygundur.
- `enum` okunabilir sabit kümeleri tanımlar; birden fazla seçeneğin birleşebildiği durumlarda `[Flags]` kullanılır. Dış kaynaklı değerler `Enum.IsDefined` ile doğrulanmalıdır.
- ValueTuple, metottan birden fazla değer döndürmenin hafif yoludur; public API'lerde `record` daha doğru bir seçimdir.
- `int?` değer tiplerine "değer yok" anlamı kazandırır. NRT açıkken `string?` ile `string` arasındaki fark derleyici tarafından denetlenir. C# 14 ile `?.` atamanın sol tarafında da kullanılabilir.
- Stack hızlı ve metot ömrüne bağlıdır; heap'teki nesneler erişilemez olduklarında generational GC tarafından toplanır. 85.000 bayt üzeri nesneler LOH'a gider.
- Boxing gizli heap ayırmalarına yol açar; generic koleksiyonlar bu maliyeti ortadan kaldırır.
- Yönetilmeyen kaynaklar `IDisposable` ile deterministik olarak serbest bırakılır; `using` ve `await using` bunu güvenli hâle getirir.

Anahtar Kavramlar

Terim	Türkçe karşılığı	Kısa açıklama
value type	değer tipi	Değişkenin verinin kendisini taşıdığı tip (`struct`, `enum`, `int`)
reference type	referans tipi	Değişkenin nesnenin adresini taşıdığı tip (`class`, `string`, dizi)
immutable	değişmez	Oluşturulduktan sonra durumu değiştirilemeyen nesne
enum	numaralandırma	İsimlendirilmiş sabitlerden oluşan değer tipi
tuple	demet	Birden fazla değeri tek yapıda taşıyan hafif tip
deconstruction	ayrıştırma	Tuple/record elemanlarını ayrı değişkenlere açma
nullable	boş olabilen	`null` değerini alabilen tip (`int?`, `string?`)
stack frame	yığın çerçevesi	Bir metot çağrısına ait parametre ve yerel değişkenler
managed heap	yönetilen öbek	GC tarafından yönetilen, nesnelerin yaşadığı bellek alanı
garbage collector	çöp toplayıcı	Erişilemeyen nesnelerin belleğini geri kazanan mekanizma
boxing / unboxing	kutulama / kutudan çıkarma	Değer tipinin `object`'e dönüştürülmesi ve geri alınması
finalizer	sonlandırıcı	GC nesneyi toplamadan önce çağrılan özel metot (`~Sinif()`)
unmanaged resource	yönetilmeyen kaynak	GC'nin tanımadığı dosya, soket, bağlantı gibi kaynaklar

Alıştırmalar

1. `Sicaklik` adında, `Derece` (double) ve `Birim` (enum: `Celsius`, `Fahrenheit`) içeren bir `readonly record struct` yazın. İki sıcaklığı birime dikkat ederek karşılaştıran bir metot ekleyin.
2. Aynı alanlara sahip bir `class` ve bir `struct` tanımlayın. Her ikisini de bir metoda parametre olarak geçirip metodun içinde alanlarını değiştirin; çıktıların neden farklı olduğunu açıklayın.

3. `[Flags]` ile bir `HaftaninGunleri` enum'ı tanımlayın. `HaftaIci` ve `HaftaSonu` birleşik değerlerini oluşturup verilen bir günün hafta sonuna düşüp düşmediğini kontrol eden metodu yazın.
4. Bir sayı dizisinin ortalamasını, standart sapmasını ve medyanını tek seferde döndüren, isimlendirilmiş elemanlara sahip bir tuple metodu yazın ve sonucu deconstruction ile kullanın.
5. NRT açık bir projede `Adres? Adres` property'sine sahip bir `Musteri` sınıfı oluşturun. Şehir bilgisini güvenle yazdıran kodu önce `if` kontrolleriyle, sonra `?.` ve `??` operatörleriyle yazın. Son olarak C# 14 null-conditional assignment ile `musteri?.Adres = ...` atamasını deneyin.
6. `ArrayList` ve `List<int>` ile 1.000.000 elemanlık döngüler kurun. `GC.CollectionCount(0)` değerini döngülerden önce ve sonra yazdırarak boxing'in GC üzerindeki etkisini gözlemleyin.
7. Bir log dosyasına yazan `DosyaLogger` sınıfını `IDisposable` olarak tasarlayın. Dispose edildikten sonra yazma denemesinde `ObjectDisposedException` fırlattığını bir test koduyla gösterin.

Ek Okuma Kaynakları

- Yapı türleri (struct) — C# başvurusu
- Class ile struct arasında seçim yapma — Tasarım yönergeleri
- Tuple türleri — C# başvurusu
- Nullable reference types
- Garbage collection temelleri
- Dispose metodu uygulama
- C# 14'teki yenilikler